

**FICHA DE COMPONENTE CURRICULAR**

CÓDIGO: [informe o código, se houver]	COMPONENTE CURRICULAR: INTRODUÇÃO A SISTEMAS PARALELOS E DISTRIBUÍDOS		
UNIDADE ACADÊMICA OFERTANTE: Faculdade de Engenharia Elétrica			SIGLA: FEELT
CH TOTAL TEÓRICA: 30 horas	CH TOTAL PRÁTICA: 15 horas	CH TOTAL A DISTÂNCIA: 0 horas	CH TOTAL: 45 horas

1. OBJETIVOS

Capacitar o estudante a entender os fundamentos de computação paralela e sistemas distribuídos, compreendendo as arquiteturas, os modelos de programação, as métricas de desempenho e as técnicas de sincronização. Ao final, espera-se que seja capaz de desenvolver aplicações básicas em MPI, OpenMP e sockets, analisando resultados e identificando desafios e oportunidades em clusters, nuvens e outras tecnologias.

2. EMENTA

Conceitos de sistemas paralelos e distribuídos. Arquiteturas (SISD, SIMD, MIMD, clusters, nuvens). Modelos de programação paralela (troca de mensagens, memória compartilhada). Análise de desempenho (speedup, eficiência, Lei de Amdahl). Sincronização, exclusão mútua e coordenação. Programação distribuída (sockets, RPC) e noções de tolerância a falhas. Projetos práticos unindo teoria e aplicação real.

3. PROGRAMA**1. Fundamentos de Sistemas Paralelos e Distribuídos****1.1. Motivação e Histórico**

- Evolução do hardware, Lei de Moore e o surgimento do paralelismo.
- Conceito de sistemas distribuídos e fatores que motivam a distribuição (escalabilidade, disponibilidade, etc.).

1.2. Arquiteturas e Modelos Computacionais

- Classificações de Flynn (SISD, SIMD, MIMD).
- Memória compartilhada vs. memória distribuída.
- Threads vs. processos: comunicação e sincronização.

2. Programação Paralela com MPI e OpenMP

2.1. Modelo de Troca de Mensagens (MPI)

- Conceitos básicos: envio e recebimento de mensagens (ponto a ponto), coletivas simples (broadcast, scatter, gather).
- Estrutura de um programa MPI: inicialização, comunicação, finalização.
- Exemplos: soma de vetores, cálculo de π (método da integração ou Monte Carlo).

2.2. Modelo de Memória Compartilhada (OpenMP/Pthreads)

- Criação de threads e seções paralelas.
- Primitivas de sincronização: semáforos, mutexes, barreiras.
- Exemplo prático: paralelização de loops e operações de redução.

3. **Algoritmos, Desempenho e Sincronização**

3.1. Algoritmos Paralelos Clássicos

- Soma de vetores, matriz-vetor, Sieve of Eratosthenes (busca de números primos).
- Comparação entre versões sequenciais e paralelas.

3.2. Métricas de Desempenho

- Speedup, eficiência, Lei de Amdahl e Lei de Gustafson.
- Granularidade e escalabilidade.

3.3. Sincronização e Exclusão Mútua

- Conceitos de corrida (race conditions), deadlocks e starvation.
- Princípios básicos de exclusão mútua em sistemas paralelos e distribuídos.

4. **Projeto 1 - Aplicação Paralela**

4.1. Definição e Escopo

- Desenvolvimento de um programa paralelo (via MPI e/ou OpenMP) para resolver um problema de maior complexidade (por exemplo, multiplicação de matrizes, ordenação paralela ou outra aplicação numérica).

4.2. Fases do Projeto

- Planejamento: análise do problema, identificação de oportunidades de paralelismo.
- Implementação: uso de rotinas MPI/OpenMP, testes de corretude e desempenho.
- Avaliação: medições de tempo, cálculo de speedup e eficiência.

4.3. Apresentação e Discussão

- Relatório curto ou apresentação em sala, enfatizando soluções

adotadas e resultados obtidos.

5. **Arquiteturas e Programação Distribuída**

5.1. Comunicação em Sistemas Distribuídos

- Sockets (TCP/UDP): conceitos de servidor/cliente, comunicação síncrona vs. assíncrona.
- RPC/RMI (visão geral): chamada de procedimentos remotos e integração entre processos.

5.2. Arquiteturas de Sistemas Distribuídos

- Cluster Computing (ex. Beowulf).
- Breve introdução a Cloud/Grid Computing (IaaS, PaaS, SaaS) e HPC.
- Topologias de interconexão (barramento, malha, anel, hipercubo).

5.3. Exemplos de Aplicações e Serviços Distribuídos

- Visão geral de sistemas de arquivos distribuídos (NFS, GFS).
- Noções de MapReduce/Hadoop, se houver tempo.

6. **Projeto 2 - Aplicação Distribuída**

6.1. Definição e Escopo

- Desenvolvimento de um serviço distribuído simples que envolva troca de mensagens em rede (via sockets ou RPC). Por exemplo: um chat distribuído, serviço de contagem distribuída ou mini-sistema de arquivos distribuído.

6.2. Fases do Projeto

- Planejamento: identificação do protocolo de comunicação, fluxos de mensagens e possíveis problemas de concorrência.
- Implementação: desenvolvimento do serviço, tratamento de erros e sincronização entre nós.
- Testes: verificação do funcionamento em ambiente multi-processo / multi-hospedeiro.

6.3. Apresentação e Discussão

- Demonstração prática do serviço em funcionamento.
- Análise de limitações e perspectivas de escalabilidade.

7. **Tópicos Avançados e Perspectivas (Seminários, textos e/ou palestras)**

7.1. Tolerância a Falhas em Sistemas Distribuídos

- Conceitos gerais, replicação de dados.
- Protocolos de commit (2PC) e noções de consenso (Paxos/Raft em alto nível).

7.2. Outras Tendências

- Computação heterogênea (CPU+GPU), breve visão de CUDA/OpenCL.
- Edge/Fog Computing, segurança e criptografia em ambientes distribuídos.
- Exascale HPC e desafios futuros.

4. COMPETÊNCIA

DECLARAÇÃO DE COMPETÊNCIA

Capacitar os alunos a projetar, desenvolver e implementar aplicações paralelas e distribuídas, utilizando modelos de programação como MPI, OpenMP e sockets. Promover habilidades para análise de desempenho, gerenciamento de sincronização e exploração de arquiteturas modernas, como clusters e computação em nuvem. Preparar os estudantes para resolver problemas computacionais de forma escalável e eficiente.

ELEMENTOS DE CONHECIMENTO | NÍVEL DE HABILIDADE

Computação Paralela e Distribuída [4.1]	Aplicação
Redes de Computadores [4.1]	Aplicação
Resolução de Problemas e Solução de Erros [4.2]	Aplicação
Comunicação Oral e Apresentação [4.2]	Aplicação

DISPOSIÇÕES

Adaptável	Inventivo	Colaborativo
-----------	-----------	--------------

5. BIBLIOGRAFIA BÁSICA

1. TANENBAUM, Andrew S. **Sistemas distribuídos**: princípios e paradigmas. São Paulo: Prentice Hall, 2007. 402 p., il. Inclui bibliografia e índice. ISBN 9788576051428 (broch.).
2. PACHECO, Peter S. **Parallel programming with MPI**. San Francisco: Morgan Kaufmann, c1997. xxii, 418 p., il. Inclui bibliografia e índice. ISBN 1558603395.
3. ROS, Alberto. **Parallel and Distributed Computing**. IntechOpen, 2010. eBook (PDF) ISBN 9789535159094. Disponível em: <https://www.intechopen.com/books/3770>.

6. BIBLIOGRAFIA COMPLEMENTAR

1. XAVIER, C. **Introduction to parallel algorithms**. New York: John Wiley & Sons, 1998. 365 p., il. (Wiley series on parallel and distributed computing). ISBN 0471251828 (enc.).
2. CHAPMAN, Barbara. **Using OpenMP**: portable shared memory parallel programming. Cambridge: MIT Press, c2008. xxii, 353 p., ill., 23 cm. (Scientific and engineering computation). Includes bibliographical references (p. [331]-347) and index. ISBN 9780262533027 (paperback : alk. paper). Disponível em: <http://www.loc.gov/catdir/toc/ecip0721/2007026656.html>. Acesso em: 18 jan. 2025.
3. SANDERS, Jason. **CUDA by example**: an introduction to general-purpose GPU programming. Upper Saddle River: Addison-Wesley, 2011. xix, 290 p., ill., 23 cm. ISBN 9780131387683 (pbk. : alk. paper).
4. COULOURIS, George F. **Distributed systems**: concepts and design. 2nd ed Reading: Addison-Wesley, 1994. xix, 644p., il. Inclui bibliografia e índice. ISBN

0201624338 (enc.).

5. CHICHLON, Joel M. **An introduction to distributed and parallel computing**. New York: PH, 1988. 209 p., il.
6. AMORIM, Claudio L. de. **Uma introdução a computação paralela e distribuída**. Campinas: Ed. da UNICAMP, 1988. 258 p.
7. HWANG, Kai. **Advanced computer architecture**: parallelism, scalability, programmability. New York: McGraw-Hill, 1993. 771p., il. (McGraw-Hill computer science series. Computer organization and architecture. Networks, parallel and distributed computing. McGraw-Hill computer engineering).). Inclui bibliografia e índice. ISBN 0070316228 : (Enc.).
8. CULLER, David E. **Parallel computer architecture**: a hardware/software approach. San Francisco: Morgan Kaufman, 1999. 1025p., il. Inclui bibliografia e índice. ISBN 1558603433 (enc.).

7. APROVAÇÃO

IGOR SOUSA PERETTA

Coordenador do Curso de Graduação em
Engenharia de Computação
Portaria R. nº 3674/2023

[nome]

Diretor(a) da Faculdade de Engenharia
Elétrica

Referência: Processo nº 23117.075354/2024-81

SEI nº 6029298